

Guia de Especificaciones para Codex

Valdris: El Nucleo Profundo

Ficha de implementacion completa — 50 clases — Java 21 + JavaFX

- * Este documento contiene una ficha por cada clase del juego con atributos, metodos, dependencias y tests.
- * Dar a Codex UNA ficha a la vez. No varias juntas. Esperar a que genere y compile antes de pasar a la siguiente.
- * Orden obligatorio: respetar las capas (1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7). Cada capa depende de la anterior.
- * Nunca usar java.util.* para listas, colas, pilas o grafos. Usar SIEMPRE las implementaciones propias.
- * Tras cada clase generada: compilar, ejecutar los tests indicados, y solo entonces pasar a la siguiente.

Estructura de paquetes del proyecto

```
src/  
MisEstructurasDeDatos/ <- reutilizar del repo anterior (LSE, Cola, Grafo)  
Valdris/  
model/  
enums/ <- CellType, EffectType, ItemType, CharacterType, EnemyType, Phase  
items/ <- Item, Weapon, Armor, Potion, Accessory  
units/ <- Unit, Player, Enemy  
map/ <- Cell, Room, Dungeon  
effects/ <- Effect  
logic/  
bfs/ <- BFSMovimiento, BFSCaminoMinimo  
vision/ <- LineaDeVision  
combat/ <- CombatManager  
ai/ <- IAEnemigo, ArbolDecisionIA  
turn/ <- TurnManager  
generation/ <- DungeonGenerator, ItemGenerator  
persistence/ <- GameState, LectorJSON  
exceptions/ <- InvalidMoveException, InvalidAttackException, GameStateException  
ui/  
MainApp.java  
model/ <- GameModel  
view/ <- GameView, CharacterSelectView, InventoryView, CombatLogView  
controller/ <- GameController  
test/  
Valdris/ <- Tests JUnit5 espejando la estructura de src
```

Reglas de dependencia entre capas

Capa	Puede usar	NO puede usar
Capa 2 — Modelo base	Solo Java puro (String, int, boolean, Math)	Nada de otras capas
Capa 3 — Unidades	Capa 2 + LSE[Effect] + LSE[Item]	Logica, UI, persistencia

Capa 4 — Mapa	Capas 2+3 + LSE[Enemy]	Logica, UI, persistencia
Capa 5 — Logica	Capas 2+3+4 + Cola + Grafo	UI, persistencia
Capa 6 — Persistencia	Capas 2+3+4+5 + Gson	UI (JavaFX)
Capa 7 — JavaFX	Todas las capas anteriores	—

CAPA 1 — Estructuras de Datos (reutilizar del repo)

- * Copiar directamente del repositorio anterior: ListaSimplementeEnlazada, Cola, Grafo, NodoGrafo, Arista.
- * NO modificar nada. Solo verificar que compilan en el nuevo proyecto.
- * Verificar que Grafo[DN,DA] tiene: addNodo, addArista, getVecinos, caminoMinimo (BFS), Dijkstra.
- * Si falta algun metodo en Grafo, aniadirlo antes de continuar con las demas capas.

CAPA 2 — Modelo base

2.1. CellType

Capa 2 — Enum

Paquete: **Valdris.model.enums**

Enum que define todos los tipos posibles de celda en la sala. Determina si una celda es transitable o no y como interactua con el jugador.

```
public enum CellType {
    FLOOR, // suelo normal, transitable
    WALL, // pared, no transitable nunca
    DOOR, // puerta abierta, transitable
    DOOR_HIDDEN, // puerta oculta, parece WALL hasta que se descubre
    DOOR_LOCKED, // puerta cerrada con llave, no transitable sin llave equipada
    STAIRS, // escaleras, cambian de piso en minas de Karath
    RUNE, // runa del suelo, transitable, activa mecanismo al pisarla
    LEVER, // palanca, transitable, se activa con accion de uso
    TRAP // trampa oculta, transitable, activa efecto al pisarla
}
```

2.2. EffectType

Capa 2 — Enum

Paquete: **Valdris.model.enums**

Enum de efectos de estado que pueden afectar a unidades (jugador y enemigos).

```
public enum EffectType {
    SLOW, // reduce mov a ceil(mov/2.0). Duracion: 2 turnos
    BLIND, // reduce mov a ceil(mov/2.0). Duracion: 2 turnos
    CURSE, // +3 daño recibido por turno. Duracion: 2 turnos
    PARALYSIS, // sin movimiento ni ataque. Duracion: 1 turno
    BURN // +3 daño al inicio del turno del afectado. Duracion: 1 turno
}
```

2.3. ItemType + CharacterType + EnemyType + Phase

Capa 2 — Enums

Paquete: **Valdris.model.enums**

Cuatro enums auxiliares. Pedir a Codex que los genere todos en un mismo mensaje.

```
public enum ItemType { WEAPON, ARMOR, SHIELD, ACCESSORY, POTION }

public enum CharacterType { Kael, SYRA, DORATH }

public enum EnemyType {
    WARRIOR, BERSERKER, GUARDIAN, // familia Warrior
}
```

```

ARCHER, SNIPER, // familia Archer
DESTRUCTOR, CONTROLLER, SUMMONER // familia Mage
}

public enum Phase { MOVEMENT, PICKUP, USE_ITEM, ATTACK, ENEMY_TURN }

```

2.4. Effect

Capa 2 — Clase

Paquete: **Valdris.model.effects**

Representa un efecto de estado activo sobre una unidad. Se almacena en LSE[Effect] dentro de Unit. Al final de cada turno se decrementa turnosRestantes. Si llega a 0 se elimina de la lista.

Tipo	Atributo	Descripcion
EffectType	tipo	Tipo de efecto (SLOW, BLIND, etc.)
int	turnosRestantes	Turnos que quedan hasta que el efecto desaparece

Firma del metodo	Descripcion	Retorna
Effect(EffectType tipo, int turnos)	Constructor. Inicializa tipo y turnosRestantes.	void
decrementar()	Resta 1 a turnosRestantes.	void
isExpired()	Devuelve true si turnosRestantes <= 0.	boolean
getTipo()	Getter de tipo.	EffectType
getTurnos()	Getter de turnosRestantes.	int

2.5. Item (abstracta)

Capa 2 — Clase abstracta

Paquete: **Valdris.model.items**

Clase base de todos los items del juego. Weapon, Armor, Potion y Accessory extienden esta clase. El metodo use() es abstracto y cada subclase lo implementa con su efecto especifico.

Tipo	Atributo	Descripcion
String	id	Identificador unico del item (ej: "W1", "P2", "AC3")
String	nombre	Nombre visible en UI (ej: "Espada Oxidada")
ItemType	tipo	Tipo de item para categorizar en el inventario
String	descripcion	Texto descriptivo para tooltip en UI

Firma del metodo	Descripcion	Retorna
Item(String id, String nombre, ItemType tipo, String desc)	Constructor base.	void
abstract void use(Player player)	Aplica el efecto del item al jugador. Implementado en subclases.	void
getId(), getNombre(), getTipo(), getDescripcion()	Getters estandar.	varios
toString()	Devuelve "[tipo] nombre" para mostrar en UI.	String

2.6. Weapon

Capa 2 — Clase

Paquete: **Valdris.model.items**

Arma equipable. El daño base REEMPLAZA el daño base del personaje (no se suma). La afinidad es un mapa CharacterType -> int con el bonus/penalizacion por personaje. use(player) equipa el arma en el slot de mano principal del jugador.

Tipo	Atributo	Descripcion
int	danoBase	Daño base del arma antes de afinidad
int	penetracion	Puntos de defensa enemiga que ignora (0 si ninguno)
Map[CharacterType,Int]	afinidades	Bonus (+) o penalizacion (-) por personaje. Usar LSE de pares o array indexado por CharacterType.ordinal()
EffectType	efectoEspecial	Efecto que puede aplicar al atacar (null si ninguno)
double	probEfecto	Probabilidad 0.0-1.0 de aplicar el efecto especial
int	rango	Rango de ataque (1=cuerpo, 3=arco, 4=largo)

Firma del metodo	Descripcion	Retorna
Weapon(String id, String nombre, int danoBase, int pen, int rango)	Constructor base.	void
setAfinidad(CharacterType tipo, int bonus)	Establece el bonus de afinidad para un personaje.	void
getAfinidad(CharacterType tipo)	Devuelve el bonus de afinidad. 0 si no tiene.	int
getDanoEfectivo(CharacterType tipo)	Devuelve danoBase + getAfinidad(tipo).	int
use(Player player)	Equipa el arma en player.setArmaEquipada(this).	void
tryAplicarEfecto()	Lanza Math.random(). Si < probEfecto devuelve efectoEspecial, sino null.	EffectType

2.7. Armor

Capa 2 — Clase

Paquete: **Valdris.model.items**

Armadura o escudo equipable. Incrementa la defensa del jugador mientras esta equipada. Puede tener una inmunidad a un efecto de estado especifico. use(player) equipa la armadura en el slot correspondiente (torso o mano secundaria).

Tipo	Atributo	Descripcion
int	defensa	Puntos de defensa que aporta al jugador
EffectType	inmunidad	Efecto al que es inmune el portador (null si ninguno)
boolean	esEscudo	true si va al slot de mano secundaria, false si va al torso

Firma del metodo	Descripcion	Retorna
Armor(String id, String nombre, int defensa, boolean esEscudo)	Constructor base.	void
use(Player player)	Equipa en slot torso o mano secundaria segun esEscudo.	void
getDefensa(), isEscudo(), getInmunidad()	Getters estandar.	varios

2.8. Potion

Capa 2 — Clase

Paquete: **Valdris.model.items**

Pocion consumible. use(player) aplica la curacion y el efecto extra si tiene. Tras usarse se elimina del inventario del jugador. La curacion no puede superar el hpMax del jugador.

Tipo	Atributo	Descripcion
int	curacionHP	HP que recupera al usarse
EffectType	efectoExtra	Efecto adicional que aplica (null si solo cura)
int	valorEfecto	Valor del efecto (ej: +5 ataque para Elixir). 0 si no aplica

Firma del metodo	Descripcion	Retorna
Potion(String id, String nombre, int curacionHP)	Constructor base.	void
use(Player player)	player.curar(curacionHP). Luego aplicar efectoExtra si tiene. Eliminar del inventario.	void

2.9. Accessory

Capa 2 — Clase

Paquete: **Valdris.model.items**

Accesorio equipable en el slot de accesorio. Efecto pasivo mientras esta equipado (bonus ataque, bonus mov, o efecto narrativo). Los accesorios AC1-AC4 son de progresion narrativa y no tienen efecto de combate directo.

Tipo	Atributo	Descripcion
int	bonusAtaque	Bonus de ataque mientras equipado (0 si ninguno)
int	bonusMov	Bonus de movimiento mientras equipado (0 si ninguno)
int	bonusDef	Bonus de defensa mientras equipado (0 si ninguno)
boolean	esNarrativo	true si es item de progresion (llave, fragmento, etc.)
String	efectoNarrativo	Descripcion del efecto especial narrativo (null si combate)

Firma del metodo	Descripcion	Retorna
Accessory(String id, String nombre)	Constructor base.	void
use(Player player)	Equipa el accesorio en player.setAccesorioEquipado(this).	void
getBonusAtaque(), getBonusMov(), getBonusDef()	Getters estandar.	varios

CAPA 3 — Unidades

3.1. Unit (abstracta)

Capa 3 — Clase abstracta

Paquete: **Valdris.model.units**

Clase base de Player y Enemy. Contiene todos los stats comunes y la gestion de efectos de estado. Los efectos se guardan en LSE[Effect]. Al inicio de cada turno de la unidad se llama a procesarEfectos().

Tipo	Atributo	Descripcion
int	hp	HP actual
int	hpMax	HP maximo. Nunca cambia durante la partida
int	ataqueBase	Daño base sin arma ni modificadores
int	defensaBase	Defensa base sin armadura
int	movBase	Puntos de movimiento base
int	rango	Rango de ataque base
int	filaActual	Posicion fila en la sala actual
int	colActual	Posicion columna en la sala actual
LSE[Effect]	efectosActivos	Lista de efectos de estado activos

Firma del metodo	Descripcion	Retorna
recibirDanio(int cantidad)	hp = max(0, hp - cantidad). Lanzar GameStateException si hp < 0 (no deberia pasar).	void
curar(int cantidad)	hp = min(hpMax, hp + cantidad).	void
isVivo()	Devuelve hp > 0.	boolean
addEfecto(Effect ef)	Anadir a efectosActivos. Si ya existe el mismo tipo, reemplazar.	void
tieneEfecto(EffectType tipo)	true si efectosActivos contiene un Effect del tipo dado.	boolean
procesarEfectos()	Aplicar efecto CURSE/BURN si activos (danio). Luego decrementar todos. Eliminar los expirados.	void
getMovEfectivo()	Si SLOW o BLIND activos: ceil(movBase/2.0). Sino: movBase.	int
getDefensaTotal()	defensaBase + bonus de armadura equipada.	int
getAtaqueTotal()	ataqueBase del arma equipada + afinidad + bonus de accesorio.	int
setPosicion(int fila, int col)	Actualiza filaActual y colActual.	void

3.2. Player

Capa 3 — Clase

Paquete: **Valdris.model.units**

El personaje controlado por el jugador. Extiende Unit. Tiene inventario (LSE[Item]) y 4 slots de equipo (arma, escudo/secundaria, torso, accesorio). Los stats base dependen del CharacterType elegido al inicio.

Tipo	Atributo	Descripcion
CharacterType	tipo	Personaje elegido: KAE, SYRA o DORATH
LSE[Item]	inventario	Todos los items que lleva el jugador
Weapon	armaEquipada	Arma activa en mano principal (null si ninguna)
Armor	escudoEquipado	Escudo o arma secundaria (null si ninguno)

Armor	armaduraEquipada	Armadura en torso (null si ninguna)
Accessory	accesorioEquipado	Accesorio activo (null si ninguno)
boolean	haMovido	true si ya uso la accion de movimiento este turno
boolean	haRecogido	true si ya uso la accion de recogida este turno
boolean	haUsadoItem	true si ya uso la accion de item este turno
boolean	haAtacado	true si ya uso la accion de ataque este turno

* Stats base por personaje:

* Kael: hpMax=110, ataqueBase=18, defensaBase=0, movBase=3, rango=1

* SYRA: hpMax=75, ataqueBase=12, defensaBase=0, movBase=5, rango=3

* DORATH: hpMax=80, ataqueBase=14, defensaBase=0, movBase=2, rango=4

Firma del metodo	Descripcion	Retorna
Player(CharacterType tipo)	Constructor. Inicializa stats segun tipo. Crea LSE vacia para inventario.	void
addItem(Item item)	Añadir item al inventario.	void
removeItem(Item item)	Eliminar item del inventario.	void
equip(Item item)	Llamar a item.use(this). Actualiza el slot correspondiente.	void
setArmaEquipada(Weapon w)	Actualiza armaEquipada. Si null, el jugador ataca con ataqueBase.	void
getAtaqueTotal()	Si armaEquipada != null: armaEquipada.getDanoEfectivo(tipo) + bonusAccesorio. Sino: ataqueBase.	int
getDefensaTotal()	defensaBase + (armadura != null ? armadura.getDefensa() : 0) + (escudo != null ? escudo.getDefensa() : 0) + bonusDef accesorio.	int
getRangoEfectivo()	Si armaEquipada != null: armaEquipada.getRango(). Sino: rango base del personaje.	int
resetAcciones()	haMovido = haRecogido = haUsadoItem = haAtacado = false. Llamar al inicio de cada turno del jugador.	void
tieneInmunidad(EffectType t)	true si armaduraEquipada o escudoEquipado tienen inmunidad a t.	boolean

3.3. Enemy

Capa 3 — Clase

Paquete: **Valdris.model.units**

Enemigo controlado por la IA. Extiende Unit. Los stats base se inicializan segun EnemyType en el constructor. dropItem es el item que suelta al morir (puede ser null si no tiene drop).

Tipo	Atributo	Descripcion
EnemyType	tipo	Tipo de enemigo que determina sus stats y comportamiento
Item	dropItem	Item que suelta al morir. Null si no tiene drop.
int	turnosSinActuar	Contador para habilidades con cooldown (Francotirador: cooldown 2t, Invocador: invoca c/2t)
String	idSala	ID de la sala donde fue generado (para Guardian: zona fija)
int	filaSpawn	Fila de spawn original (para Guardian: calcular zona fija radio 3)
int	colSpawn	Columna de spawn original
boolean	esMiniJefe	true si es mini-boss con stats reforzados

* Stats por tipo (ver tabla completa en guia_diseno_v5):

* WARRIOR: hp=35, atk=15, def=8, mov=2, rng=1

* BERSERKER: hp=25, atk=13, def=3, mov=4, rng=1

* GUARDIAN: hp=50, atk=15, def=10, mov=1, rng=1

* ARCHER: hp=28, atk=10, def=4, mov=3, rng=4

* SNIPER: hp=28, atk=18, def=3, mov=2, rng=5

* DESTRUCTOR: hp=40, atk=6, def=5, mov=0, rng=5 (AOE, daño por celda)

* CONTROLLER: hp=35, atk=4, def=4, mov=2, rng=3

* SUMMONER: hp=45, atk=0, def=6, mov=2, rng=0

Firma del metodo	Descripcion	Retorna
Enemy(EnemyType tipo, int fila, int col, String idSala)	Constructor. Inicializa stats segun tipo. Guarda posicion de spawn.	void
onDeath(Room room)	Llamar cuando hp <= 0. Si dropltem != null, colocarlo en la celda actual de la sala.	void
incrementarCooldown()	turnosSinActuar++. Para Sniper y Summoner.	void
resetCooldown()	turnosSinActuar = 0.	void
isCooldownListo(int n)	Devuelve turnosSinActuar >= n.	boolean

CAPA 4 — Mapa

4.1. Cell

Capa 4 — Clase

Paquete: **Valdris.model.map**

Representa una celda individual de la sala. Contiene su tipo, la unidad que la ocupa (si hay alguna) y el ítem en el suelo (si hay alguno). `isWalkable()` es el método más importante: determina si una unidad puede moverse a esta celda.

Tipo	Atributo	Descripción
CellType	tipo	Tipo de celda
Unit	unit	Unidad que ocupa la celda (null si vacía)
Item	item	Ítem en el suelo (null si no hay)
boolean	descubierta	false si es DOOR_HIDDEN y no ha sido revelada aun

Firma del método	Descripción	Retorna
Cell(CellType tipo)	Constructor. unit=null, item=null, descubierta=true (excepto DOOR_HIDDEN).	void
isWalkable()	false si tipo==WALL o tipo==DOOR_LOCKED. false si unit!=null. false si item!=null && item.isRequiresAdjacent(). true en el resto.	boolean
setUnit(Unit u)	Establece la unidad en la celda.	void
removeUnit()	unit = null.	void
setItem(Item i)	Coloca un ítem en el suelo.	void
removeItem()	item = null. Devuelve el ítem que había.	Item
revelar()	Si tipo==DOOR_HIDDEN: descubierta=true. La celda pasa a comportarse como DOOR.	void
getTipo(), getUnit(), getItem(), isDescubierta()	Getters estándar.	varios

4.2. Room

Capa 4 — Clase

Paquete: **Valdris.model.map**

Sala del juego. Contiene una matriz de `Cell[][]` y la lista de enemigos vivos. El tamaño varía según el tipo (Pequeña 5-6x5-6, Mediana 7-8x7-9, Grande 9-10x9-11). Los pasillos de transición son siempre 3x8.

Tipo	Atributo	Descripción
String	id	Identificador único (ej: "S1-A", "S3-F", "PASILLO-1-2")
String	nombre	Nombre visible (ej: "Aldea Abandonada")
Cell[][]	celdas	Matriz de celdas. celdas[filas][cols]
int	filas	Número de filas de la sala
int	cols	Número de columnas de la sala
LSE[Enemy]	enemigos	Lista de enemigos vivos en la sala
boolean	hasRoomTimer	true si la sala tiene límite de turnos
int	turnosRestantes	Turnos restantes si hasRoomTimer==true. -1 si no tiene timer
boolean	explorada	true si el jugador ya ha entrado alguna vez
int	filaJugador	Fila donde aparece el jugador al entrar a esta sala
int	colJugador	Columna donde aparece el jugador al entrar a esta sala

Firma del metodo	Descripcion	Retorna
Room(String id, String nombre, int filas, int cols)	Constructor. Inicializa celdas[][] con FLOOR. Crea LSE vacia para enemigos.	void
getCell(int fila, int col)	Devuelve celdas[fila][col]. Lanzar InvalidMoveException si fuera de rango.	Cell
setCellType(int f, int c, CellType t)	Cambia el tipo de una celda.	void
addEnemigo(Enemy e)	Añadir enemigo a la lista y colocarlo en su celda.	void
removeEnemigo(Enemy e)	Eliminar enemigo de la lista y limpiar su celda.	void
getEnemigos()	Devuelve LSE[Enemy] de enemigos vivos.	LSE[Enemy]
isEnRango(int f, int c)	true si $0 \leq f < \text{filas}$ y $0 \leq c < \text{cols}$.	boolean
decrementarTimer()	Si hasRoomTimer: turnosRestantes--. Si llega a 0 lanzar GameStateException("tiempo agotado").	void
getCeldaLibreCercana(int f, int c)	Busca la celda libre mas cercana a (f,c) para spawn de Berserker invocado.	Cell

4.3. Dungeon

Capa 4 — Clase

Paquete: **Valdris.model.map**

El mapa completo del juego. Usa Grafo[Room,String] para conectar las salas. El grafo es bidireccional excepto la arista Pasillo Final -> S5-D que es unidireccional. roomActual es la sala donde esta el jugador en este momento.

Tipo	Atributo	Descripcion
Grafo[Room,String]	grafo	Grafo de salas. Nodos=Room, Aristas=String (descripcion conexion)
Room	roomActual	Sala donde esta el jugador actualmente
NodoGrafo[Room]	nodoActual	Nodo del grafo correspondiente a roomActual

Firma del metodo	Descripcion	Retorna
Dungeon()	Constructor. Inicializa grafo vacio.	void
addRoom(Room room)	Crea NodoGrafo[Room] y lo añade al grafo.	void
conectar(Room a, Room b, String desc)	Conecta bidireccional. grafo.addArista(nodoA, desc, nodoB) y viceversa.	void
conectarUnidireccional(Room a, Room b, String desc)	Solo grafo.addArista(nodoA, desc, nodoB). Para Pasillo Final -> S5-D.	void
getSalasAdyacentes(Room r)	Devuelve LSE[Room] con las salas conectadas a r (vecinos en el grafo).	LSE[Room]
getRoomById(String id)	Busca en el grafo la sala con ese id. Null si no existe.	Room
setRoomActual(Room r)	Actualiza roomActual y nodoActual.	void
getRoomActual()	Getter de roomActual.	Room

CAPA 5 — Logica del juego

5.1. BFSMovimiento

Capa 5 — Clase
estatica

Paquete: **Valdris.logic.bfs**

Calcula las celdas alcanzables por una unidad dado su punto de origen y sus puntos de movimiento. Usa BFS en 4 direcciones (N, S, E, O). Las diagonales cuestan 2 pasos. Una celda es alcanzable si `isWalkable() == true` para todas las celdas del camino.

Firma del metodo	Descripcion	Retorna
getCellsInRange(Room room, int filaOrigen, int colOrigen, int movPoints)	BFS desde (filaOrigen, colOrigen). Expande en 4 dir. Solo añade celda si <code>isWalkable()</code> . Devuelve <code>LSE[Cell]</code> con todas las celdas alcanzables (sin incluir la de origen).	<code>LSE[Cell]</code>
getCamino(Room room, int filaOrigen, int colOrigen, int filaDestino, int colDestino)	BFS para encontrar el camino mas corto entre dos celdas. Devuelve <code>LSE[Cell]</code> con el camino (incluye origen y destino). <code>LSE</code> vacia si no hay camino.	<code>LSE[Cell]</code>

* Implementacion BFS: usar Cola donde `int[] = {fila, col, pasosUsados}`.

* Mantener una matriz `boolean[][]` visitado del mismo tamaño que la sala.

* Solo expandir celdas con `pasosUsados < movPoints`.

* La celda destino del jugador puede tener un enemigo (para calcular rango de ataque): NO incluirla en celdas alcanzables de movimiento.

5.2. LineaDeVision

Capa 5 — Clase
estatica

Paquete: **Valdris.logic.vision**

Implementa el algoritmo de Bresenham para determinar si hay linea de vision directa entre dos celdas. Usado por Archer, Sniper, Destructor y Controller para decidir si pueden atacar.

Firma del metodo	Descripcion	Retorna
tieneVision(Room room, int f1, int c1, int f2, int c2)	Algoritmo de Bresenham entre (f1,c1) y (f2,c2). Devuelve true si todas las celdas intermedias son de tipo FLOOR, DOOR o RUNE (no WALL). Las celdas de origen y destino no se comprueban.	<code>boolean</code>

```
// Pseudocodigo Bresenham para referencia de Codex:
int dx = abs(c2-c1), dy = abs(f2-f1);
int sx = c1<c2 ? 1 : -1, sy = f1<f2 ? 1 : -1;
int err = dx - dy;
int f = f1, c = c1;
while (!(f==f2 && c==c2)) {
    int e2 = 2*err;
    if (e2 > -dy) { err -= dy; c += sx; }
    if (e2 < dx) { err += dx; f += sy; }
    if (f==f2 && c==c2) break;
    if (room.getCell(f,c).getTipo()==WALL) return false;
}
return true;
```

5.3. CombatManager

Capa 5 — Clase
estatica

Paquete: **Valdris.logic.combat**

Gestiona toda la logica de combate. Formula oficial: $\text{danio} = \max(0, \text{ataque} * \text{random}[0.5-1.5] - \text{defensaEfectiva})$. La defensa efectiva del defensor se reduce por la penetracion del arma del atacante.

Firma del metodo	Descripcion	Retorna
calcularDanio(Unit atacante, Unit defensor)	aleatorio = Math.random()*1.0+0.5. ataqueTotal = atacante.getAtaqueTotal(). pen = arma.getPenetracion() si tiene. defEf = max(0, defensor.getDefensaTotal()-pen). return max(0, (int)(ataqueTotal*aleatorio) - defEf).	int
resolverAtaqueJugador(Player jugador, Enemy enemigo)	Calcular danio. enemigo.recibirDanio(danio). Si arma tiene efecto especial: tryAplicarEfecto() y si no null addEfecto al enemigo. Si !enemigo.isVivo(): enemigo.onDeath(room).	int
resolverAtaqueEnemigo(Enemy enemigo, Player jugador)	Calcular danio. Si jugador.tieneEfecto(CURSE): danio+=3. jugador.recibirDanio(danio). Devolver danio real infligido.	int
resolverAOEDestructor(Enemy destructor, Room room, Player jugador)	Calcular celdas en radio 2 alrededor del destructor. Por cada celda: si hay jugador, infligir destructor.getDanoBase() directamente (sin formula aleatoria). Devolver danio total.	int
estaEnRango(Unit atacante, Unit defensor)	Calcular distancia Manhattan entre posiciones. Si <= atacante.getRangoEfectivo(): true. Para arcos/magicos: ademas verificar LineaDeVision.tieneVision().	boolean

5.4. ArbolDecisionIA

Capa 5 — Clase

Paquete: **Valdris.logic.ai**

Arbol de decision que determina la accion de un enemigo en su turno. Implementado como arbol binario de condiciones. Cada nodo interno es una condicion boolean. Las hojas son acciones (MOVER, ATACAR, USAR_ESPECIAL, ESPERAR). Este arbol cubre el criterio de evaluacion de arboles del enunciado.

* Estructura del arbol por tipo de enemigo:

* WARRIOR/BERSERKER: ¿puede atacar? -> ATACAR. ¿puede moverse hacia jugador? -> MOVER. Sino ESPERAR.

* GUARDIAN: ¿jugador en zona fija (radio 3 del spawn)? -> comportarse como Warrior. Sino ESPERAR.

* ARCHER/SNIPER: ¿tiene vision Y en rango? -> ATACAR. ¿jugador fuera zona confort? -> MOVER_A_ZONA. Sino ESPERAR.

* DESTRUCTOR: ¿jugador en radio 2? -> AOE. Sino ESPERAR (no se mueve).

* CONTROLLER: ¿tiene vision Y en rango? -> APLICAR_EFECTO. Sino MOVER. Sino ESPERAR.

* SUMMONER: ¿cooldown listo? -> INVOCAR_BERSERKER. Sino MOVER (huir del jugador).

Tipo	Atributo	Descripcion
NodoArbol	raiz	Raiz del arbol de decision para el tipo de enemigo

Firma del metodo	Descripcion	Retorna
ArbolDecisionIA(EnemyType tipo)	Constructor. Construye el arbol de decision para ese tipo de enemigo.	void
decidirAccion(Enemy enemy, Room room, Player player)	Recorre el arbol desde la raiz evaluando condiciones. Devuelve la AccionIA de la hoja alcanzada.	AccionIA

```
// Clase interna NodoArbol:
class NodoArbol {
    Condition condicion; // funcion boolean (Enemy, Room, Player) -> boolean
    NodoArbol siTrue; // rama si condicion es verdadera
    NodoArbol siFalse; // rama si condicion es falsa
    AccionIA accion; // null si es nodo interno, AccionIA si es hoja
```

```

boolean isHoja() { return accion != null; }

}

// Enum AccionIA:
enum AccionIA { ATACAR, MOVER, MOVER_A_ZONA, APLICAR_EFECTO, INVOCAR, AOE, ESPERAR }

```

5.5. IAEnemigo

Capa 5 — Clase
estatica

Paquete: **Valdris.logic.ai**

Ejecuta el turno de un enemigo usando ArbolDecisionIA para decidir la accion y BFSCaminoMinimo para calcular el movimiento. executeTurn() es el metodo principal llamado por TurnManager para cada enemigo.

Firma del metodo	Descripcion	Retorna
executeTurn(Enemy enemy, Room room, Player player, CombatManager cm)	Llamar a procesarEfectos(). Si PARALYSIS activo: return. Obtener accion de ArbolDecisionIA. Ejecutar la accion correspondiente.	void
ejecutarMovimiento(Enemy enemy, Room room, Player player)	BFSCaminoMinimo desde posicion enemigo hasta posicion jugador. Mover enemy tantos pasos como enemy.getMovEfectivo() a lo largo del camino.	void
ejecutarAtaque(Enemy enemy, Player player, CombatManager cm)	Si estaEnRango: CombatManager.resolverAtaqueEnemigo(). Si CONTROLLER: aplicar efecto aleatorio (SLOW/BLIND/CURSE) en lugar de daño.	void
invocarBerserker(Enemy summoner, Room room)	Crear nuevo Enemy(BERSERKER). Colocarlo en celda libre cercana al Summoner. Añadirlo a room.getEnemigos(). summoner.resetCooldown().	void

5.6. TurnManager

Capa 5 — Clase

Paquete: **Valdris.logic.turn**

Gestiona el ciclo de turnos del juego. Orden de fases del jugador: MOVEMENT -> PICKUP -> USE_ITEM -> ATTACK -> ENEMY_TURN. El jugador puede ceder el turno en cualquier momento pasando a ENEMY_TURN. Despues de ENEMY_TURN siempre viene MOVEMENT del jugador.

Tipo	Atributo	Descripcion
Phase	faseActual	Fase actual del turno
Dungeon	dungeon	Referencia al dungeon actual
Player	player	Referencia al jugador
int	turnoGlobal	Contador de turnos totales. Siempre incrementa.
GameModel	modelo	Referencia al GameModel de JavaFX para notificar cambios (puede ser null en tests)

Firma del metodo	Descripcion	Retorna
TurnManager(Dungeon dungeon, Player player)	Constructor. faseActual=MOVEMENT. turnoGlobal=0.	void
ejecutarMovimiento(int filaDestino, int colDestino)	Verificar que faseActual==MOVEMENT y !player.haMovido. Verificar celda en BFS range. Mover jugador. Si celda es DOOR/STAIRS: changeRoom(). Si celda tiene item en suelo: popup (notificar modelo). player.haMovido=true. Avanzar a PICKUP.	void
ejecutarRecogida()	Verificar PICKUP y !player.haRecogido. Buscar contenedor adyacente (4 dirs). Si hay: ofrecer recogida. player.haRecogido=true. Avanzar a USE_ITEM.	void

ejecutarUsoltem(Item item)	Verificar USE_ITEM y !player.haUsadoItem. player.equip(item) o item.use(player). player.haUsadoItem=true. Avanzar a ATTACK.	void
ejecutarAtaque(Enemy objetivo)	Verificar ATTACK y !player.haAtacado. Verificar estaEnRango. CombatManager.resolverAtaqueJugador(). player.haAtacado=true. Avanzar a ENEMY_TURN.	void
cederTurno()	Avanzar directamente a ENEMY_TURN independientemente de la fase actual.	void
ejecutarTurnoEnemigos()	Verificar ENEMY_TURN. turnoGlobal++. decrementarTimer sala si aplica. Para cada enemy en sala: !AEenemigo.executeTurn(). procesarEfectos() del jugador. player.resetAcciones(). faseActual=MOVEMENT. Notificar modelo.	void
changeRoom(Room destino)	dungeon.setRoomActual(destino). Colocar jugador en destino.filaJugador/colJugador. destino.explorada=true.	void

5.7. DungeonGenerator

Capa 5 — Clase
estatica

Paquete: **Valdris.logic.generation**

Genera el Dungeon completo con todas las salas, conexiones, enemigos e items. El tamaño exacto de cada sala se elige aleatoriamente dentro del rango del tipo. Los subtipos de enemigo se eligen aleatoriamente dentro de cada familia. El resultado se serializa a JSON inmediatamente para persistencia.

* Salas a generar (12 salas jugables + 4 pasillos = 16 en total):

* Z1: S1-A(7x8 med), S1-B(5x6 peq), S1-C(9x10 gr), S1-D(9x9 gr), S1-SEC(5x5 peq), PASILLO-1-2(3x8)

* Z2: S2-A(6x6 peq), S2-B(7x8 med), S2-C(9x10 gr), S2-D(5x6 peq), S2-E(10x10 gr), S2-SEC(5x5 peq), PASILLO-2-3(3x8)

* Z3: S3-A(7x7 med), S3-SEC(5x6 peq) -- version reducida para 7 dias

* Conexiones: ver grafo en guia_proyecto_v3. La unica arista unidireccional es PASILLO-FINAL -> S5-D.

Firma del metodo	Descripcion	Retorna
generarMundo()	Crea todas las salas, las conecta en el Dungeon, genera enemigos e items, devuelve Dungeon listo.	Dungeon
generarSala(String id, String nombre, int filas, int cols, boolean hasTimer)	Crea Room. Rellena bordes con WALL. Coloca DOOR en posiciones de conexion. Devuelve Room.	Room
poblarEnemigos(Room room, LSE specs)	Para cada EnemySpec: elegir subtipo aleatorio, crear Enemy, colocarlo en posicion aleatoria libre.	void
colocarItem(Room room, Item item, int fila, int col)	room.getCell(fila,col).setItem(item).	void
elegirSubtipo(String familia)	Familia "warrior" -> aleatorio entre WARRIOR/BERSERKER/GUARDIAN. "archer" -> ARCHER/SNIPER. "mage" -> DESTRUCTOR/CONTROLLER/SUMMONER.	EnemyType

5.8. ItemGenerator

Capa 5 — Clase
estatica

Paquete: **Valdris.logic.generation**

Fabrica de items. Genera cualquier item del juego por ID o por zona. Contiene los stats de todos los items de la guia de disenio v5.

Firma del metodo	Descripcion	Retorna
crearItem(String id)	Devuelve el item correspondiente al ID (W1-W12, A1-A8, P1-P5, AC1-AC8) con todos sus stats configurados.	Item

itemAleatorioZona(int z)	Devuelve un ítem aleatorio del pool de la zona z (1-4). Ver sección 4.7 de la guía.	Item
crearDropEnemigo(EnemyType tipo)	Según probabilidad del tipo (ver tabla 4.6): devuelve Item o null.	Item

* Todos los stats de ítems están en la guía_diseno_v5.pdf sección 4.2-4.5.

* crearItem() debe ser un switch/if-else exhaustivo con todos los IDs. No usar reflection.

* Los ítems son objetos nuevos cada vez (no singletons). Cada llamada crea una instancia nueva.

CAPA 6 — Persistencia

6.1. Excepciones personalizadas

Capa 6 — Clases

Paquete: **Valdris.exceptions**

Tres excepciones personalizadas para cubrir los casos de error del juego. Todas extienden Exception (checked). Pedir a Codex que las genere todas en un mensaje.

```
// Las tres en el mismo mensaje a Codex:
public class InvalidMoveException extends Exception {
    public InvalidMoveException(String message) { super(message); }
}

public class InvalidAttackException extends Exception {
    public InvalidAttackException(String message) { super(message); }
}

public class GameStateException extends Exception {
    public GameStateException(String message) { super(message); }
}
```

* InvalidMoveException: lanzar cuando el jugador intenta moverse a celda no alcanzable o fuera de rango.

* InvalidAttackException: lanzar cuando el jugador intenta atacar a enemigo fuera de rango o sin enemigo seleccionado.

* GameStateException: lanzar en errores de estado (tiempo agotado, HP negativo, sala inexistente).

6.2. GameState

Capa 6 — Clase

Paquete: **Valdris.persistence**

Snapshot serializable del estado completo del juego. Gson serializa/deserializa esta clase a JSON. Todos los campos deben ser tipos primitivos, String, arrays o listas anidadas de los mismos. Las referencias a objetos se guardan por ID (String) para evitar referencias circulares.

Tipo	Atributo	Descripcion
String	idRoomActual	ID de la sala donde esta el jugador
String	tipoPersonaje	CharacterType.name() del jugador
int	hpJugador	HP actual del jugador
String[]	itemsInventario	Array de IDs de items en el inventario
String	armaEquipada	ID del arma equipada (null si ninguna)
String	armaduraEquipada	ID de armadura equipada (null si ninguna)
String	accesorioEquipado	ID de accesorio equipado (null si ninguno)
int	turnoGlobal	Turno global actual
EnemyStateDTO[]	enemigos	Array de estados de todos los enemigos del mundo
String[]	itemsSuelo	Array de "idSala:fila:col:idItem" para items en el suelo

```
// EnemyStateDTO (clase interna estatica de GameState):
static class EnemyStateDTO {
    String idSala;
    String tipoEnemigo; // EnemyType.name()
    int fila, col;
    int hp;
```

```
boolean vivo;  
String dropItemId; // null si no tiene drop  
}
```

6.3. LectorJSON

Capa 6 — Clase
estatica

Paquete: **Valdris.persistence**

Serializacion y deserializacion del GameState usando Gson. `guardarPartida()` convierte el estado actual a GameState y lo escribe en JSON. `cargarPartida()` lee el JSON y reconstruye todas las referencias de objetos.

Firma del metodo	Descripcion	Retorna
guardarPartida(Dungeon dungeon, Player player, TurnManager tm, String rutaArchivo)	Crear GameState desde los objetos. <code>Gson.toJson(gameState, writer)</code> . Envolver en try-catch <code>IOException</code> . Lanzar <code>GameStateException</code> si falla.	void
cargarPartida(String rutaArchivo)	<code>Gson.fromJson(reader, GameState.class)</code> . Reconstruir Dungeon con <code>DungeonGenerator</code> . Restaurar estado de enemigos (hp, posicion, vivos/muertos). Restaurar inventario del jugador. Devolver array <code>[Dungeon, Player, int turnoGlobal]</code> .	Object[]
extraerGameState(Dungeon d, Player p, TurnManager tm)	Metodo auxiliar que construye el GameState desde los objetos del juego.	GameState
reconstruirDesdeGameState(GameState gs)	Metodo auxiliar que reconstruye los objetos del juego desde el GameState.	Object[]

- * CRITICO: Al deserializar, los enemigos con `vivo=false` deben eliminarse de la sala (no recrearlos).
- * Los items del inventario se recrean llamando a `ItemGenerator.crearItem(id)` para cada ID guardado.
- * Usar `try-with-resources` para `FileReader` y `FileWriter`.
- * Lanzar `GameStateException` (no `IOException`) para que la capa de UI no dependa de `java.io`.

CAPA 7 — Interfaz JavaFX (MVC)

* Patron MVC: GameModel (datos) -> GameView (visualizacion) -> GameController (eventos).

* GameModel implementa el patron Observer: GameView se registra como listener y se actualiza cuando cambia el estado.

* NUNCA llamar a logica del juego directamente desde GameView. Siempre pasar por GameController.

* La UI solo lee datos de GameModel. Nunca modifica Player, Room o Enemy directamente.

7.1. MainApp

Capa 7 — JavaFX

Paquete: **Valdris.ui**

Punto de entrada de la aplicacion JavaFX. start() muestra primero CharacterSelectView. Cuando el jugador elige personaje, crea GameModel, GameController y GameView y cambia la escena.

```
public class MainApp extends Application {
    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("Valdris: El Nucleo Profundo");
        primaryStage.setWidth(1280);
        primaryStage.setHeight(720);
        primaryStage.setResizable(false);
        // Mostrar pantalla de seleccion de personaje
        CharacterSelectView selectView = new CharacterSelectView(primaryStage);
        primaryStage.setScene(selectView.getScene());
        primaryStage.show();
    }
    public static void main(String[] args) { launch(args); }
}
```

7.2. GameModel

Capa 7 — MVC Model

Paquete: **Valdris.ui.model**

Contiene referencias a todos los objetos del juego y notifica a los listeners cuando algo cambia. Es el puente entre la logica (capas 1-6) y la vista (JavaFX). Los listeners son tipicamente GameView y CombatLogView.

Tipo	Atributo	Descripcion
Dungeon	dungeon	El dungeon actual
Player	player	El jugador
TurnManager	turnManager	El gestor de turnos
LSE[GameModelListener]	listeners	Lista de objetos que escuchan cambios
String	ultimoMensaje	Ultimo mensaje del log de combate

Firma del metodo	Descripcion	Retorna
GameModel(CharacterType tipo)	Constructor. DungeonGenerator.generarMundo(). new Player(tipo). new TurnManager(dungeon, player). this.	void
addListener(GameModelListener l)	Anadir listener a la LSE.	void
notificarCambio()	Para cada listener en LSE: listener.onEstadoCambiado(this).	void
notificarMensaje(String msg)	ultimoMensaje = msg. notificarCambio().	void

getDungeon(), getPlayer(), getTurnManager()	Getters estandar.	varios
--	-------------------	--------

```
// Interfaz listener (en el mismo paquete):
public interface GameModelListener {
    void onEstadoCambiado(GameModel modelo);
}
```

7.3. GameView

Capa 7 — MVC View

Paquete: **Valdris.ui.view**

Vista principal del juego. Layout BorderLayout: Centro=GridPane con el mapa de la sala, Derecha=panel lateral (HP, inventario rapido, fase actual), Inferior=CombatLogView. Se actualiza al recibir notificacion de GameModel.

Tipo	Atributo	Descripcion
BorderPane	root	Layout raiz de la vista
GridPane	gridSala	Grid donde se renderizan las celdas de la sala
VBox	panelLateral	Panel derecho con stats del jugador
Label	labelHP	Muestra HP actual / HP maximo
Label	labelFase	Muestra la fase actual del turno
HBox	barraRapida	4-5 slots de items equipados y pociones
CombatLogView	logCombate	Panel inferior con mensajes de combate
GameModel	modelo	Referencia al modelo
GameController	controller	Referencia al controlador

Firma del metodo	Descripcion	Retorna
GameView(GameModel modelo, GameController controller)	Constructor. Construir layout. modelo.addListener(this).	void
onEstadoCambiado(GameModel modelo)	Llamado por GameModel. Redibujar sala, actualizar panel lateral, actualizar barra rapida.	void
renderizarSala(Room room)	Limpiar gridSala. Para cada celda: crear Rectangle 40x40px con color segun CellType. Añadir sprites de unidades e items encima.	void
resaltarCeldas(LSE[Cell] celdas)	Añadir overlay verde transparente a las celdas alcanzables por BFS.	void
limpiarResultado()	Eliminar todos los overlays de resaltado.	void
getScene()	Devuelve new Scene(root, 1280, 720).	Scene

* Colores de celdas: FLOOR=gris claro, WALL=gris oscuro, DOOR=marron, DOOR_HIDDEN=igual que WALL, TRAP=rojo muy oscuro.

* Sprites de unidades: rectangulos de color. Kael=azul, Syra=verde, Dorath=morado, enemigos=rojo.

* Items en suelo: punto amarillo pequeno sobre la celda.

* Celda resaltada (BFS): overlay verde 30% opacidad.

* Cada Rectangle del grid tiene setMouseClicked() que notifica al GameController.

7.4. GameController

Capa 7 — MVC Controller

Paquete: **Valdris.ui.controller**

Recibe todos los eventos de la UI y los traduce a llamadas a TurnManager. Es el unico que llama a metodos de TurnManager. Maneja las excepciones InvalidMoveException e InvalidAttackException mostrando mensajes en el log.

Firma del metodo	Descripcion	Retorna
onCeldaClick(int fila, int col)	Segun fase actual: MOVEMENT -> turnManager.ejecutarMovimiento(f,c) catch InvalidMoveException. ATTACK -> buscar enemigo en celda y turnManager.ejecutarAtaque(enemigo) catch InvalidAttackException.	void
onBotonCederTurno()	turnManager.cederTurno(). turnManager.ejecutarTurnoEnemigos().	void
onBotonGuardar()	LectorJSON.guardarPartida(). modelo.notificarMensaje("Partida guardada.").	void
onBotonCargar()	LectorJSON.cargarPartida(). Reconstruir modelo. notificarCambio().	void
onBotonInventario()	Abrir InventoryView en nueva ventana modal. Solo disponible en fase USE_ITEM.	void
onItemBarraRapidaClick(Item item)	turnManager.ejecutarUsoltem(item) si fase==USE_ITEM.	void
onBotonNuevaPartida()	Crear nuevo GameModel. Actualizar vistas.	void

7.5. CharacterSelectView

Capa 7 — Vista

Paquete: **Valdris.ui.view**

Pantalla de inicio donde el jugador elige su personaje. Muestra los 3 personajes con sus stats. Al hacer click en uno, crea el GameModel y cambia la escena principal a GameView.

Firma del metodo	Descripcion	Retorna
CharacterSelectView(Stage stage)	Constructor. Construir layout con 3 botones de personaje.	void
crearBotonPersonaje(CharacterType tipo)	VBox con nombre, stats (HP/MOV/RANGO) y boton Elegir. Al click: iniciarJuego(tipo).	VBox
iniciarJuego(CharacterType tipo)	new GameModel(tipo). new GameController(modelo). new GameView(modelo, controller). stage.setScene(view.getScene()).	void
getScene()	Devuelve la Scene de seleccion.	Scene

7.6. InventoryView

Capa 7 — Vista

Paquete: **Valdris.ui.view**

Ventana modal de inventario completo. Lado izquierdo: silueta del personaje con 4 slots de equipo marcados. Lado derecho: lista de todos los items del inventario. Al hacer click en un item: se equipa (llama a controller.onItemBarraRapidaClick()).

Firma del metodo	Descripcion	Retorna
InventoryView(GameModel modelo, GameController controller)	Constructor. Construir layout.	void
mostrarSilueta()	BorderPane con 4 Labels/Rectangles para los slots: ARMA, ESCUDO, TORSO, ACCESORIO. Mostrar nombre del item equipado en cada slot o "Vacio".	Node
mostrarInventario()	ListView o VBox con todos los items del inventario. Cada fila: nombre + tipo + boton Usar.	Node
show()	Abrir en nueva Stage modal (initModality(APPLICATION_MODAL)).	void

7.7. CombatLogView

Capa 7 — Vista

Paquete: **Valdris.ui.view**

Panel inferior que muestra los ultimos mensajes de combate. Maximo 5 mensajes visibles. Los nuevos entran por abajo y los viejos salen por arriba.

Firma del metodo	Descripcion	Retorna
CombatLogView()	Constructor. VBox con 5 Labels inicialmente vacios.	void
addMensaje(String msg)	Desplazar labels hacia arriba. Poner msg en el ultimo label.	void
onEstadoCambiado(GameModel m)	Si m.getUltimoMensaje() != null: addMensaje().	void
getNode()	Devuelve el VBox raiz para incluir en el layout principal.	Node

Tests JUnit5 — Especificaciones completas

* Escribir los tests DESPUES de que Codex genere cada clase. Los tests verifican el comportamiento, no la implementacion.

* Estructura de cada test: Arrange (preparar datos) -> Act (ejecutar metodo) -> Assert (verificar resultado).

* No testear metodos privados directamente. Testear el comportamiento publico.

* Las clases JavaFX (Capa 7) NO se testean con JUnit. Solo las capas 2-6.

T.1. CellTest

Test JUnit5

Paquete: **Valdris.model.map**

Tests unitarios para Cell. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testIsWalkable_FLOOR_sinUnidad	Cell FLOOR sin unidad -> isWalkable() == true
testIsWalkable_WALL	Cell WALL -> isWalkable() == false siempre
testIsWalkable_conUnidad	Cell FLOOR con unidad -> isWalkable() == false
testIsWalkable_DOOR_LOCKED	Cell DOOR_LOCKED -> isWalkable() == false
testRevelar_DOOR_HIDDEN	Cell DOOR_HIDDEN -> revelar() -> comportarse como DOOR
testRemoveItem_devuelveItem	Colocar item, removeItem() -> devuelve el item y la celda queda vacia

T.2. RoomTest

Test JUnit5

Paquete: **Valdris.model.map**

Tests unitarios para Room. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testConstructor_creaMatrizCorrecta	Room 7x8 -> getCell(0,0) existe, getCell(6,7) existe
testGetCell_fueraDeRango	getCell(-1,0) -> lanza InvalidMoveException
testAddEnemigo_colocaEnCelda	addEnemigo(enemy) -> getCell(f,c).getUnit() == enemy
testRemoveEnemigo_limpiaCelda	addEnemigo + removeEnemigo -> celda.getUnit() == null
testDecrementarTimer_lanzaExcepcion	Room con timer=1 -> decrementarTimer() -> GameStateException

T.3. BFSMovimientoTest

Test JUnit5

Paquete: **Valdris.logic.bfs**

Tests unitarios para BFSMovimiento. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testGetCellsInRange_salaAbierta	Sala 5x5 FLOOR, origen centro, mov=2 -> al menos 8 celdas alcanzables
testGetCellsInRange_bloqueadoPorWall	WALL entre origen y destino -> destino no en lista
testGetCellsInRange_bloqueadoPorUnidad	Unidad en celda adyacente -> esa celda no en lista
testGetCamino_encuentraCamino	Camino entre dos celdas sin obstaculos -> lista no vacia, primer elemento = origen
testGetCamino_sinCamino	Destino rodeado de WALL -> lista vacia
testGetCamino_longitudCorrecta	Camino recto de 3 celdas -> lista.getSize() == 3

T.4. LineaDeVisionTest

Test JUnit5

Paquete: **Valdris.logic.vision**

Tests unitarios para LineaDeVision. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testTieneVision_lineaRecta	Dos celdas en misma fila sin obstaculos -> true
testTieneVision_bloqueadaPorWall	WALL entre dos celdas -> false
testTieneVision_diagonal	Dos celdas en diagonal sin obstaculos -> true
testTieneVision_mismaCelda	Misma celda origen y destino -> true

T.5. CombatManagerTest

Test JUnit5

Paquete: **Valdris.logic.combat**

Tests unitarios para CombatManager. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testCalcularDanio_rangoValido	calcularDanio 1000 veces -> siempre >= 0
testCalcularDanio_conPenetracion	Arma pen=5, defensor def=4 -> def efectiva=0 -> danio=ataque*aleatorio
testCalcularDanio_defensaMayorAtaque	Ataque bajo, defensa alta -> danio == 0
testResolverAtaqueJugador_reducaHP	HP enemigo disminuye tras ataque valido
testResolverAtaqueJugador_mataEnemigo	HP enemigo <= 0 -> !enemigo.isVivo()
testEstaEnRango_dentroDe	Unidades adyacentes con rango 1 -> true
testEstaEnRango_fueraDe	Unidades a distancia 5 con rango 1 -> false

T.6. PlayerTest

Test JUnit5

Paquete: **Valdris.model.units**

Tests unitarios para Player. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testConstructor_KAEL	Player(KAEL) -> hp=110, movBase=3, rango=1
testConstructor_SYRA	Player(SYRA) -> hp=75, movBase=5, rango=3
testConstructor_DORATH	Player(DORATH) -> hp=80, movBase=2, rango=4
testCurar_noSuperaHpMax	HP=50, curar(100) -> hp==hpMax
testRecibirDanio_noNegativo	HP=10, recibirDanio(20) -> hp==0
testGetAtaqueTotal_conArmaYAfinidad	Equipar W1(Kael): ataque = 16+4 = 20
testGetDefensaTotal_conArmadura	Equipar A2(+6 def): defensaTotal >= 6
testGetMovEfectivo_conSlow	Syra (mov5) + SLOW -> getMovEfectivo() == 3
testResetAcciones	Marcar todas las acciones como usadas -> resetAcciones() -> todas false
testTieneInmunidad_escudoRaices	Equipar A3 -> tieneInmunidad(SLOW) == true

T.7. EnemyTest

Test JUnit5

Paquete: **Valdris.model.units**

Tests unitarios para Enemy. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testConstructor_WARRIOR	Enemy(WARRIOR,...) -> hp=35, ataque=15, def=8, mov=2
testConstructor_GUARDIAN	Enemy(GUARDIAN,...) -> hp=50, def=10, mov=1
testIsCooldownListo	turnosSinActuar=2, isCooldownListo(2) -> true
testOnDeath_colocaDropEnCelda	Enemy con dropltem -> onDeath(room) -> celda.getItem() != null
testOnDeath_sinDrop	Enemy sin dropltem -> onDeath(room) -> celda.getItem() == null

T.8. ItemTest

Test JUnit5

Paquete: **Valdris.model.items**

Tests unitarios para Item. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testWeapon_getDanoEfectivo_conAfinidad	W1 con Kael: getDanoEfectivo(KAEL) == 20
testWeapon_getDanoEfectivo_sinAfinidad	W1 con Syra: getDanoEfectivo(SYRA) == 16
testWeapon_use equipaEnJugador	W1.use(player) -> player.getArmaEquipada() == W1
testPotion_use curaHP	Jugador HP=50, P1.use(player) -> hp==75
testPotion_use_noSuperaMax	Jugador HP=100, hpMax=110, P2.use(player) -> hp==110
testArmor_use equipaEnSlot	A2.use(player) -> player.getArmaduraEquipada() == A2
testArmor_escudo equipaEnSecundaria	A1(escudo).use(player) -> player.getEscudoEquipado() == A1
testAccessory_use equipaEnSlot	AC5.use(player) -> player.getAccesorioEquipado() == AC5

T.9. TurnManagerTest

Test JUnit5

Paquete: **Valdris.logic.turn**

Tests unitarios para TurnManager. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testFaseInicial_MOVEMENT	new TurnManager -> faseActual == MOVEMENT
testEjecutarMovimiento_avanzaFase	Movimiento valido -> faseActual == PICKUP
testEjecutarMovimiento_celdaInvalida	Celda WALL -> lanza InvalidMoveException
testCederTurno_vaAENEMY_TURN	cederTurno() -> faseActual == ENEMY_TURN
testEjecutarTurnoEnemigos_incrementa Turno	ejecutarTurnoEnemigos() -> turnoGlobal == 1
testEjecutarAtaque_fueraDeRango	Enemigo fuera de rango -> lanza InvalidAttackException
testResetAcciones_trasEnemyTurn	Tras ejecutarTurnoEnemigos -> player.haMovido == false

T.10. GameStateTest

Test JUnit5

Paquete: **Valdris.persistence**

Tests unitarios para GameState. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
-----------------	--------------

testGuardarYCargar_hpJugador	Guardar con HP=75, cargar -> HP==75
testGuardarYCargar_roomActual	Guardar en sala S1-A, cargar -> roomActual.getId()=="S1-A"
testGuardarYCargar_inventario	Guardar con W1 en inventario, cargar -> inventario contiene W1
testGuardarYCargar_enemigoMuerto	Matar enemigo, guardar, cargar -> ese enemigo no esta en la sala
testCargar_archivoInexistente	Ruta invalida -> lanza GameStateException
testGuardar_creaArchivo	guardarPartida() -> el archivo existe en disco

T.11. DungeonGeneratorTest

Test JUnit5

Paquete: **Valdris.logic.generation**

Tests unitarios para DungeonGenerator. Ejecutar tras implementar la clase correspondiente.

Nombre del test	Que verifica
testGenerarMundo_noNull	generarMundo() -> dungeon != null
testGenerarMundo_salasSuficientes	El dungeon tiene al menos 12 salas
testGenerarMundo_salaS1A_existe	getRoomById("S1-A") != null
testGenerarMundo_conexionBidireccional	S1-A conectada a S1-B y viceversa
testGenerarMundo_enemigosEnSala	S1-A tiene al menos 1 enemigo

Instrucciones de uso con Codex

- * ORDEN OBLIGATORIO: seguir las fichas en el orden numerado. Nunca saltar capas.
- * Una ficha = un mensaje a Codex. No mezclar dos fichas en el mismo prompt.
- * Siempre empezar el prompt con: "Proyecto: Valdris El Nucleo Profundo. Java 21. Sin java.util.* para estructuras."
- * Siempre terminar el prompt con: "Usa LSE[T] en lugar de ArrayList, Cola en lugar de Queue, Grafo en lugar de Map para conexiones."
- * Tras cada clase: compilar. Si hay errores de compilacion, corregirlos antes de pasar a la siguiente ficha.
- * Tras cada clase: ejecutar los tests de esa clase. Si fallan, pedir a Codex que corrija.

Plantilla de prompt para Codex

```
Proyecto: Valdris El Nucleo Profundo. Java 21. Sin java.util.* para listas/colas/grafos.
Usa siempre: ListaSimplementeEnlazada<T>, Cola<T>, Grafo[DN,DA] (implementaciones propias).

Genera la clase [NOMBRE_CLASE] en el paquete [PAQUETE].

Atributos:
[copiar seccion de atributos de la ficha]

Metodos:
[copiar seccion de metodos de la ficha]

Notas adicionales:
[copiar info_box de la ficha si existe]

Genera tambien el test JUnit5 [NOMBRE_TEST] con estos casos:
[copiar tests de la ficha]
```

Checklist de integracion por capa

Capa	Verificar antes de pasar a la siguiente
Capa 1	LSE, Cola y Grafo compilan. Grafo.caminoMinimo() devuelve lista no vacia en test basico.
Capa 2	Todos los enums y clases base compilan. Effect.isExpired() funciona. Weapon.getDanoEfectivo() devuelve valor correcto.
Capa 3	Player(KAEL).getMovEfectivo() == 3. Player con SLOW -> getMovEfectivo() == 2. Enemy(WARRIOR).isVivo() == true.
Capa 4	Room(7,8): getCell(0,0) ok, getCell(7,8) lanza excepcion. Cell WALL -> isWalkable()===false.
Capa 5	BFS en sala 5x5 devuelve celdas correctas. CombatManager.calcularDanio() siempre >= 0. TurnManager pasa por todas las fases.
Capa 6	guardarPartida() crea archivo JSON. cargarPartida() reconstruye el mismo HP y sala. GameStateException se lanza correctamente.
Capa 7	MainApp abre ventana 1280x720. CharacterSelectView muestra 3 personajes. GameView renderiza sala. Click en celda mueve jugador.